

Claude가 Unreal MCP로 액션 데모 블루프린트를 만든 과정

소울워커식 액션 RPG 팀 프로젝트를 준비하며, 에디터 내장 Unreal MCP만으로 블루프린트를 어디까지 만들 수 있는지 확인한 기록이다. 무엇을 어떤 순서와 규칙으로 만들었는지, 어디서 막혔고 어떻게 풀었는지 정리했다.

프로젝트	ActionDemo (Third Person BP 템플릿 + Combat, Platforming 변형 팩)
엔진	Unreal Engine 5.8.3, 에디터 내장 MCP 서버 <code>http://127.0.0.1:8000/mcp</code>
작업자	Claude Code (claude-opus-5-5, 데스크톱 앱 Code 탭 로컬 세션). 준비 키트는 Claude 클라우드 세션, 플레이 검증은 사람
진행	2026-09-28 12:05 ~ 13:56 (KST), 단계 0~11(사람 플레이는 13:51 종료). 단계별 커밋 14개(보고서 제외), 같은 날 <code>main</code> 에 머지
근거 문서	<code>docs/demo/02-demo-spec.md</code> , <code>03-mcp-build-plan.md</code> , <code>04-mcp-test-log.md</code> , <code>assets.md</code>

6 / 6

완료 기준 통과
(1~5 플레이, 6 문서)

41

MCP로 만든 에셋
(블루프린트 28개 포함)

0

사람이 에디터에서
직접 고친 블루프린트

1시간 41분

단계 0 시작부터
자동 검증 끝까지

결론. 태그, 스탯 테이블, GameplayEffect, 큐, 어빌리티 그래프, 입력, UI, 맵까지 데모에 필요한 에셋 41개와 설정을 전부 MCP로 만들었다. 사람은 준비 단계(프로젝트 생성, MCP 연결 승인)와 마지막 플레이 검증을 맡았고, 에디터에서 블루프린트를 고친 일은 없다. 다만 **클래스 참조 변수**, **BP 인터페이스 추가**, **몽타주 노티파이**를 다루는 전용 툴이 없었다. **Python(unreal 모듈)**과 **PIE 키 입력**도 MCP로는 쓸 수 없었다. 그래서 설계를 우회하거나 에디터 UI 자동화를 썼고, 입력이 필요한 동작 확인은 사람이 플레이로 했다.

판정 집계 (28개 단계)

판정	단계 수	대표 예
성공	25	MCP 툴만으로 끝냄. 그중 4개는 우회를 포함한다(6-1 클래스 리터럴, 7-1 태그 기반 발동, 7-3 UI 자동화, 10-1 REINST 수정)
부분	1	1 템플릿 조사: 몽타주 섹션과 노티파이 시간을 MCP로 읽지 못함
Python	0	MCP의 Python 샌드박스는 <code>unreal</code> 모듈을 import할 수 없어 대체 경로 자체가 없었다
실패	1	10-4 사람 없이 어빌리티 발동을 자동으로 검증하는 단계. Python은 쓸 수 없었고, SlateInspector로 보낸 키도 게임에 전달되지 않았다
해당 없음	1	10-5 사람 플레이 테스트(원래 사람이 하는 단계)

판정 뜻(03 계획서 기준): **성공** 전용 MCP 툴만으로 끝냄. **Python** 전용 툴이 없어 MCP 안의 Python 실행으로 해결. **부분** 결과는 나왔지만 일부가 모자람(원래 정의는 "사람이 고쳐야 했음"이고, 조사 단계라 고칠 것이 없어 "일부 정보를 못 읽음"으로 적용). **실패** MCP로 못 해서 사람이 대신함. **해당 없음** 원래 사람이 하는 단계.

1 무엇을 만들었나

좌클릭 3타 콤보, Shift 회피(무적 0.4초), Q 돌진 베기, E 대지 강타(띄우기), R 각성(8초간 공격력 1.5배)을 쓰는 GAS 액션 데모다. 훈련용 더미 3개가 맞으면 HP 바가 줄고, HP가 0이 되면 래그돌로 쓰러졌다가 3초 뒤 부활한다. C++는 GAS 요건 때문에 `UDemoAttributeSet` 하나만 두고, 나머지는 전부 블루프린트로 만들었다.

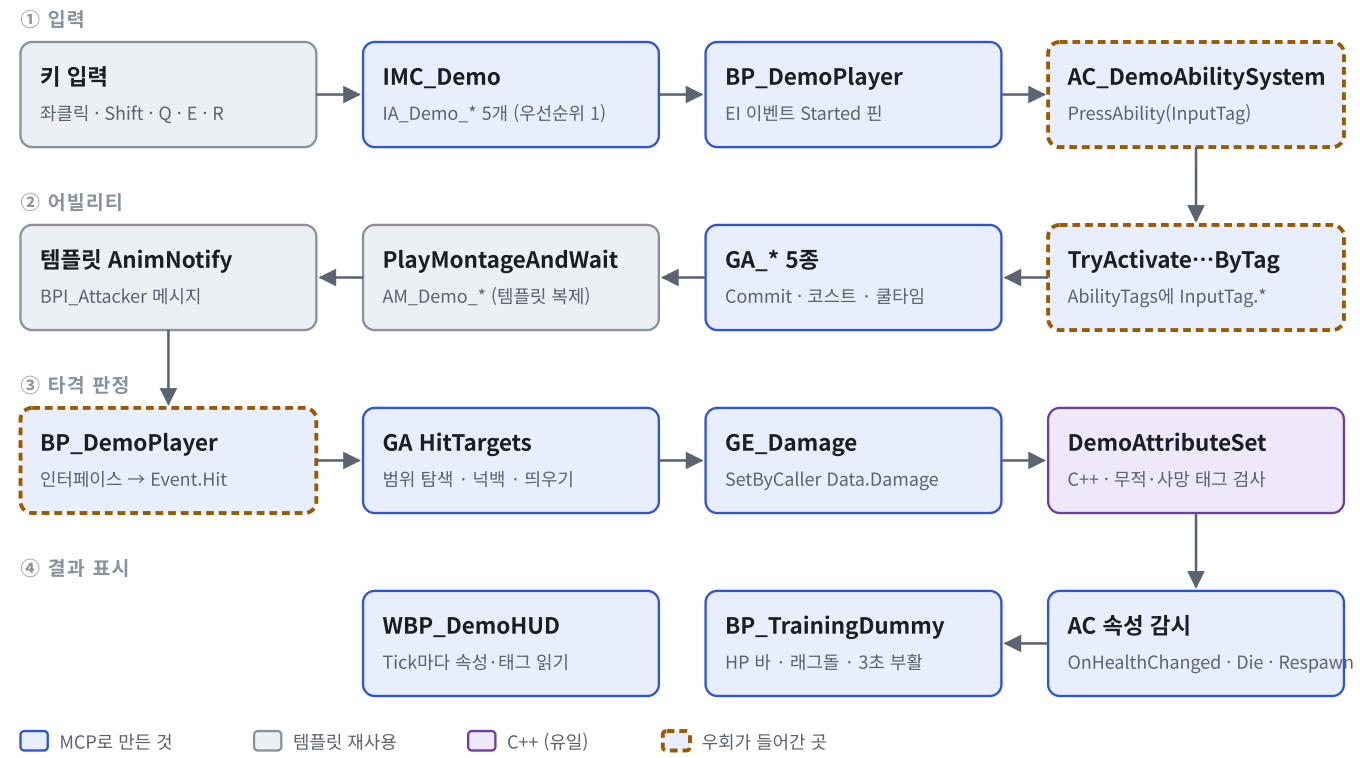


그림 1. 데모의 동작 흐름. 점선 테두리는 MCP 툴 한계 때문에 설계를 바꾼 지점이다(3장 '우회해서 됨' 카드와 4장 참고).

만든 에셋 41개 (/Game/Demo)

Abilities	GA_DemoBase, GA_Attack, GA_Dodge, GA_Skill_DashStrike, GA_Skill_GroundSlam, GA_Skill_Awaken
Effects	GE_Damage, GE_Cost_Skill1/2/3, GE_Cooldown_Dodge, GE_Cooldown_Skill1/2/3, GE_DodgeInvuln, GE_Awaken, GE_ManaRegen, GE_RestoreFull
Cues · VFX	GC_Hit, GC_Slam (Burst), GC_Awaken (Looping), NS_Demo_Slam
Anims	AN_DemoGameplayEvent, AM_Demo_Combo, AM_Demo_Slam, AM_Demo_Dodge (템플릿 몽타주 복제)
Core	AC_DemoAbilitySystem, BP_DemoGameMode
Characters	BP_DemoPlayer (BP_ThirdPersonCharacter 자식), BP_TrainingDummy
Input	IA_Demo_Attack, IA_Demo_Dodge, IA_Demo_Skill1/2/3, IMC_Demo
Data · UI · Maps	DT_Attr_Player, DT_Attr_Dummy, WBP_DemoHUD, WBP_DummyHealth, L_DemoArena

설정 변경: 게임플레이 태그 24개(`DefaultGameplayTags.ini`), 큐 경로(`DefaultGame.ini`), 기본 맵(`DefaultEngine.ini`).



그림 2. PIE 화면. 플레이어, 머리 위 HP 바가 달린 훈련용 더미 3개, 좌하단 HUD(HP·MP 바, Shift/Q/E/R 쿨타임 슬롯). SlateInspectorToolset.Screenshot으로 찍었다.

2 작업 방식

GAS Gameplay Ability System. 스킬, 수치, 상태 태그를 다루는 언리얼 플러그인 SetByCaller GE 수치를 에셋에 고정하지 않고, 적용하는 쪽이 태그 (여기서는 <code>Data.Damage</code>)로 값을 넘기는 방식 CDO Class Default Object. 클래스마다 하나씩 있는 기본값 오브젝트. REINST_ 는 재컴파일 때 옛 클래스를 옮겨 두는 임시 클래스 PIE Play In Editor. 에디터 안에서 게임을 실행하는 것	GA·GE·GC GameplayAbility(스킬 로직), GameplayEffect(수치 변경과 태그 부여), GameplayCue(이펙트 연출) DSL BlueprintTools가 제공하는 그래프 작성용 텍스트 언어. 괄호로 감싼 S-expression으로 노드와 연결을 한 번에 적는다 strict 컴파일 경고도 에러로 보는 컴파일(<code>warnings_as_errors</code>). dirty 는 저장하지 않은 변경이 있다고 표시된 상태 서브에이전트 메인 Claude 세션이 일을 나눠 맡기려고 띄우는 별도 Claude. 결과 요약만 돌려준다
--	--

2.1 전체 흐름과 역할 분담

시각(KST)	누가	무엇을
11:27	Claude (클라우드 세션, 에디터 없음)	사양(02), 작업 계획(03), 기록표 양식(04), C++ <code>DemoAttributeSet</code> , 설치 스크립트 <code>install.ps1</code> , <code>CLAUDE.md</code> 와 MCP 연결 설정을 미리 작성(커밋 f715441)
~12:03	사람 + Claude Code (PC 이전 세션)	사람이 BP 템플릿으로 <code>ActionDemo</code> 를 만들고, 함께 <code>install.ps1 -Build</code> 를 적용해 <code>DemoAttributeSet</code> 모듈과 <code>GameplayAbilities·Unreal MCP·All Toolsets</code> 플러그인을 켜다. BP 템플릿에 빠진 <code>Combat·Platforming</code> 변형 팩은 엔진 폴더에서 복사했다(커밋 c50c1ef). 모두 MCP가 아니라 셀 작업이다
12:05~13:46	Claude Code + Unreal MCP (이 세션)	계획서 단계 0~10-4를 MCP로 진행. 단계마다 기록표를 채우고 커밋

시각(KST)	누가	무엇을
~13:51	사람	PIE에서 직접 플레이해 완료 기준 1~5 확인
~13:56	Claude Code	플레이 결과와 요약을 기록하고 커밋(단계 11), 이 보고서 작성

2.2 MCP와 연결

- MCP(Model Context Protocol)는 AI가 외부 프로그램의 기능을 "툴"로 불러 쓰게 하는 공개 규약이다. UE 5.8 에디터에는 MCP 서버가 들어 있어서, 에디터가 켜지면 8000번 포트(127.0.0.1)에서 서버가 시작된다. Claude Code(Anthropic의 코딩 에이전트)는 이 서버에 HTTP로 붙어서 에셋 생성, 속성 설정, 컴파일 같은 에디터 기능을 툴 호출로 실행한다. 서버가 PC 안에서만 열리므로 Claude Code도 같은 PC에서 돌아야 한다.
- 에셋은 모두 MCP로 만들었다. git 커밋, 스크린샷 PNG 변환, 엔진 툴 소스 읽기는 Claude Code의 셸과 파일 도구로 했다.
- Claude Code는 저장소의 `.mcp.json` 으로 서버에 붙는다. **세션을 저장소 루트에서 열어야** 이 파일이 읽힌다. 첫 세션은 상위 폴더에서 열어 연결이 안 됐고, 사람이 루트에서 세션을 다시 열어 연결을 승인했다.
- MCP 툴은 처음부터 다 보이지 않고, 메타 툴 3개로 필요할 때 찾아 쓴다. `list_toolsets` 로 툴셋 52개의 목록을 보고, `describe_toolset` 으로 사용법을 읽고, `call_tool` 로 실행한다. 툴셋마다 툴이 여러 개 있고, BlueprintTools 한 툴셋에만 53개가 있다.

2.3 진행 원칙

원칙	내용
문서가 기준	클라우드 세션이 미리 써 둔 사양(02), 작업 순서(03), 규칙(CLAUDE.md)을 그대로 따랐다. 계획을 바꿀 때는 이유를 <code>assets.md</code> 와 기록표에 한 줄씩 남겼다.
단계마다 기록·커밋	단계가 끝나면 04 기록표에 사용한 툴(툴셋·함수), 판정, 걸린 시간, 메모를 적고 바로 로컬 커밋했다. 세션이 끊겨도 기록표 마지막 행부터 이어갈 수 있게 했다.
호출은 한 번에 하나	에디터 하나에는 MCP 호출을 한 번에 하나씩만 보냈다. 동시에 부르면 응답이 다른 호출과 뒤섞이기 때문이다(4장 첫 행).
추측 금지	툴 이름·인자·노드 타입·핀 이름은 반드시 조회한 뒤 썼다. 설명만으로 한계가 불분명하면 엔진 플러그인의 툴 소스(<code>EditorToolset/Content/Python</code>)를 직접 읽어 확정했다.
템플릿은 읽기만	템플릿 원본은 고치지 않고 <code>/Game/Demo</code> 로 복제해서 썼다. 저장도 <code>/Game/Demo</code> 에셋만 했다.

2.4 에셋 하나를 만드는 루프

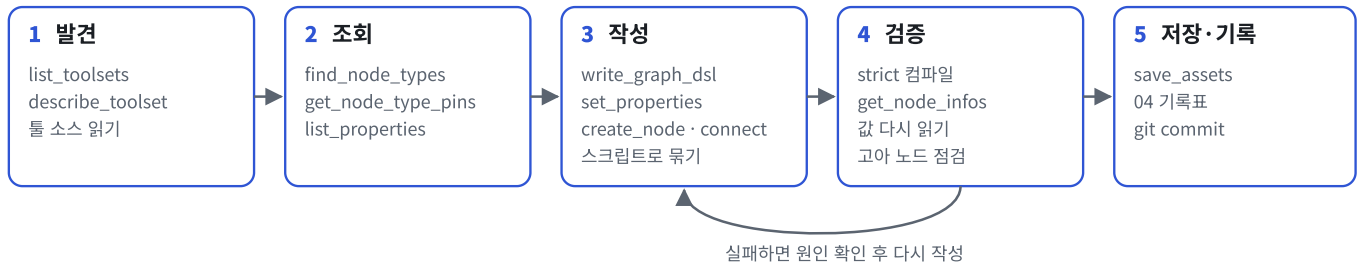


그림 3. 에셋마다 1~4를 반복하고, 저장·기록·커밋은 단계 단위로 했다. 컴파일은 strict로 돌렸다. `read_graph_dsl` 은 Enhanced Input 이벤트의 Started 핀 연결을 보여 주지 않고 pure 노드를 쓰인 곳마다 펼쳐 보여 주므로, 핵심 연결은 `get_node_infos` 로 다시 읽어 확인했다. 계획서가 경고한 "DSL이 bool 리터럴을 빠뜨리거나 고아 노드를 남긴다"는 문제는 이번에 나타나지 않았다.

2.5 툴을 쓰는 요령

- **그래프는 DSL로 한 번에.** BlueprintTools의 S-expression DSL(`write_graph_dsl`)로 함수와 이벤트 그래프를 통째로 썼다. GA_Attack 콤보 그래프는 한 번 쓰기로 노드 49개가 만들어졌다. 어빌리티 태스크의 비동기 출력 핀은 `(:OnCompleted ...)` 같은 연속 블록으로 이어 붙였다.
- **DSL로 안 되는 곳은 노드 API로.** Enhanced Input 이벤트의 `Started` 핀처럼 DSL로 고를 수 없는 연결은 `create_node` → `connect_pins` → `set_pin_value` 로 만들었다. DSL을 다시 써도 이 노드들은 지워지지 않는 것을 확인했다.
- **속성은 ObjectTools로.** GE의 모디파이어, 큐, 인스턴스드 서브오브젝트 배열(GE 컴포넌트), IMC 매핑, 위젯 슬롯 레이아웃, 컴포넌트 템플릿 값 모두 `set_properties` 로 넣었다. 넣은 뒤에는 `get_properties` 로 다시 읽어 비교했다.
- **반복은 ProgrammaticToolset로 묶기.** 태그 24개(단계 2), GE 10개(단계 4), 입력 이벤트 노드 15개(7-4)처럼 반복되는 호출은 짧은 Python 스크립트 하나로 묶어 왕복 횟수를 줄였다. 이 샌드박스는 툴 호출만 묶을 수 있어서 판정은 "성공"으로 기록했다.
- **큰 출력은 파일로.** `describe_toolset` 결과(최대 약 278KB)는 파일로 받아 필요한 부분만 추렸다. 스크린샷 base64(약 1.8MB)는 스크립트 안에서 바로 파일로 저장해 컨텍스트를 아꼈다.

2.6 서브에이전트 활용

말긴 일	방식	결과
템플릿 조사(단계 1)	플레이어·입력, Combat 변형, Platforming·Niagara를 조사 에이전트 3개에게 나눴다. 처음엔 병렬로 돌렸지만 응답 뒤섞임을 발견하고 중단했다. 이후 한 명씩 순차로 돌리고, 플레이어와 Combat 조사의 핵심 주장 28개는 별도 검증 에이전트가 MCP로 다시 확인했다.	28개 주장 모두 유지. 결과는 <code>assets.md</code>
BP 인터페이스 추가(7-3)	SlateInspectorToolset로 에디터 UI를 직접 조작해 Class Settings → Implemented Interfaces → Add → BPI_Attacker를 눌렀다.	성공. 스크린샷 없이 위젯 트리만으로 처리
PIE 입력 진단(10-4)	게임 뷰포트를 클릭하고 키를 보낸 뒤 바로 인스펙터로 효과를 확인했다.	키가 게임에 전달되지 않음을 확정

스냅샷, 툴 설명처럼 출력이 큰 작업을 서브에이전트에게 맡기면 메인 세션의 컨텍스트를 아낄 수 있다. 에이전트가 MCP를 쓰는 동안 메인 세션은 호출을 멈췄다.

3 단계별 진행과 결과

시간은 커밋 간격 기준이다. 단, 단계 0은 12:05 세션 시작부터 췄고, 10-5는 PIE 종료 시각(13:51)으로 추정했다. 서브에이전트가 템플릿을 조사하는(단계 1) 동안 단계 2(태그)를 먼저 끝냈다.

단계	내용	주로 쓴 툴	판정	소요	
0	연결, 툴셋·DSL 문법 확인	<code>list_toolsets</code> , <code>describe_toolset</code> , <code>get_graph_dsl_docs</code>	성공	4분	
2	게임플레이 태그 24개	<code>GameplayTagsToolset.AddTag</code>	성공	3분	
1	템플릿 조사(읽기 전용)	<code>AssetTools</code> , <code>BlueprintTools</code> , <code>ObjectTools</code> , <code>NiagaraToolset_System</code>	부분	38분	
3	스탯 DataTable 2개	<code>DataTableTools</code>	성공	1분	
4	GE 12개, 큐 3개, Niagara 1개	<code>BlueprintTools.create</code> , <code>ObjectTools</code> , <code>GameplayCueToolset</code> , <code>NiagaraToolset_System</code>	성공	6분	

단계	내용	주로 쓴 툴	판정	소요	
5	AnimNotify(함수 오버라이드), 몽타주 복제	add_function_graph, write_graph_dsl, AssetTools.duplicate	성공	1분	
6	GA 6개(기반 함수 5개, 콤보, 회피, 스킬 3종)	write_graph_dsl, set_pin_value, compile_blueprint	성공 우회	11분	
7-8	AC, 입력 에셋, 플레이어, 더미, UMG 2개	ActorTools.add_component, UMGToolSet, SlateInspector, create_node	성공 우회	16분	
9	게임 모드, 맵, 기본 맵 설정	SceneTools, ConfigSettingsToolset	성공	2분	
10- 1~3	전체 컴파일, PIE, 스크린샷, GAS 인스펙터	compile_blueprint, StartPIE, AbilitySystemInspector	성공 수정 1	19분	
10-4	사람 없이 어빌리티 발동 자동 검증(Python 불가, 대안으로 보낸 키도 게임에 안 들어감)	SlateInspector.PressKey/Click	실패		
10-5	사람 플레이 테스트(완료 기준 1~5)	LogsToolset(오류 수집)	해당 없음	약 5분	

완료 기준

#	기준	확인 방법	결과
1	좌클릭 연타로 콤보가 이어지고 더미 HP 바가 줄어든다	사람 플레이	통과
2	Q, E, R이 마나를 쓰고 쿨타임 동안 HUD 슬롯이 어두워진다	사람 플레이	통과
3	E를 맞은 더미가 공중에 뜬다	사람 플레이	통과
4	회피 중에는 데미지를 받지 않는다	사람 플레이	통과
5	더미가 죽으면 3초 뒤 살아난다	사람 플레이	통과
6	04 기록표에 모든 단계의 결과가 있다	문서 확인(28개 행 모두 판정)	통과

자동 검증으로는 부여된 어빌리티 5개, 속성 500/100/20, GE_ManaRegen 적용, 더미 HP 1000을 인스펙터로 확인했다. 플레이 구간 로그에는 Blueprint Runtime Error, Accessed None, ensure가 없었다.

MCP로 된 것과 안 된 것

MCP 전용 툴로 됨

- 태그, DataTable, GE 모디파이어와 SetByCaller, GE 컴포넌트(인스턴스드 서브오브젝트 배열)
- 큐 노티파이 생성과 부모 변경, 템플릿에서 Niagara 시스템 생성
- BP 그래프: 함수, 함수 오버라이드, 어빌리티 태스크, 커스텀 이벤트, 디스패처, 컴포넌트 바인딩 이벤트
- 변수(기본형, 구조체, 오브젝트, 배열, 로컬), 컴포넌트 추가
- IA, IMC 매핑(defaultKeyMappings.mappings), UMG 트리와 레이아웃
- 맵 복제와 액터 배치, WorldSettings, 프로젝트 설정, PIE 시작·종료, GAS 인스펙터, 로그

우회해서 됨

- 클래스 참조 변수·파라미터 → 클래스 리터럴 핀, InputTag 기반 발동
- BP 인터페이스 추가 → SlateInspectorToolset UI 자동화
- 새 레벨 생성 → 엔진 템플릿 맵 Template_Default 복제
- enum 배열 리터럴 → DSL로 빈 MakeArray 연결 후 set_pin_value
- 태그 컨테이너 리터럴 → MakeGameplayTagContainerFromTag 노드
- PIE 스크린샷 → SlateInspector.Screenshot 을 파일로 저장

사람이 함

- BP 템플릿 프로젝트 생성, 설치 스크립트 적용 확인(준비 단계, 이전 Claude Code 세션과 함께)
- 저장소 루트에서 Claude Code 세션 다시 열기(MCP 연결)
- 플레이 테스트로 입력, 콤보, 무적, 부활 확인
- 에디터에서 블루프린트를 직접 고친 일은 없음

MCP로 못 함

- 몽타주 섹션·노티파이 읽기와 쓰기(CompositeSections, Notifies)
- Python unreal 모듈 사용(샌드박스 허용 모듈: json, math, re, time, datetime, copy)
- PIE 게임 뷰포트에 키 입력 보내기
- GameplayCueToolset으로 새로 만든 큐 조회(에디터 큐 라이브러리가 갱신되지 않음. 플레이에서는 문제 보고 없음)

4 막힌 곳과 풀 방법

다음 작업자가 같은 곳에서 시간을 쓰지 않도록, 증상과 원인과 대응을 정리했다.

증상	원인	대응
동시 호출하자 응답이 다른 호출의 결과로 돌아옴	에디터 하나에 붙은 MCP 서버에 여러 요청이 동시에 들어감(실제 실행은 됨)	에디터당 호출 1개. 서브에이전트도 순차로만 실행
계획서의 "Python 대체"가 불가	ProgrammaticToolset은 툴 호출을 묶는 용도라 unreal 모듈을 import할 수 없음	전용 툴 조합과 UI 자동화로 대체. 안 되면 사람에게 넘김
클래스 참조 변수를 만들 수 없음. UClass 오브젝트 변수는 타입이 integer로 깨짐	클래스 참조(TSubclassOf) 타입을 만드는 변수 툴이 없음(add_variable 은 기본형 7종과 구조체 5종, add_struct_variable 은 구조체, add_object_variable 은 오브젝트 참조만)	클래스 리터럴 핀으로 쓰고, 발동은 AbilityTags의 InputTag로

증상	원인	대응
첫 PIE에서 어빌리티가 <code>REINST_GA_Attack_C_5</code> 로 부여되고 ensure 발생	클래스 변수 대신 GA·GE의 CDO를 오브젝트 참조로 넣었다. BP를 재컴파일하면 엔진은 옛 CDO를 REINST_ 임시 클래스로 옮기고 새 CDO를 만든다. 그래서 옛 CDO를 가리키던 참조에서 GetClass를 부르면 REINST_ 클래스가 나온다. 클래스 참조는 원래 클래스를 가리키므로 영향이 없다	CDO 참조를 모두 클래스 리터럴로 교체. GA를 다시 컴파일한 뒤에도 정상임을 확인
몽타주 섹션과 노티파이를 읽을 수 없음	ObjectTools가 해당 속성을 노출하지 않음	템플릿 노티파이가 <code>BPI_Attacker</code> 메시지를 보낸다는 점을 이용해, 노티파이 추가 대신 인터페이스 구현으로 설계 변경
BP 인터페이스를 추가할 툴이 없음	ObjectTools는 Blueprint를 항상 CDO로 바꿔 다룸	SlateInspector로 Class Settings UI 조작. Details 목록을 따로 Observe해야 Add 버튼이 잡힘
DSL 작성 실패	멤버 변수는 맨 이름으로 못 읽음, bool 변수 <code>bIsDead</code> 의 게터는 <code>GetIsDead</code> , 태그 컨테이너 리터럴 불가, <code>+</code> 는 리터럴이 붙어야 문자열 연결	노드 타입을 먼저 조회. 실패한 쓰기는 스냅샷 롤백돼 부분 노드가 남지 않음
괄호가 든 이벤트 이름을 DSL에 쓸 수 없음	<code>OnHealthChanged(DemoAbilitySystem)</code> 의 괄호를 토큰라이저가 분리	이벤트 이름을 따옴표로 감쌘
읽기만 했는데 템플릿 BP 12개가 dirty	<code>find_node_types</code> , <code>get_node_type_pins</code> 가 내부적으로 임시 노드를 만들었다 지움	템플릿은 저장하지 않는 것이 원칙이다. 이번에는 에디터를 끌 때 12개가 다시 저장됐지만 <code>.gitignore</code> 대상이라 저장소에는 영향이 없다
새로 만든 큐를 GameplayCueToolset이 못 찾음	에디터 큐 라이브러리가 큐 생성 전에 초기화됨	에셋 태그(<code>GameplayCueName</code>)로 등록을 확인하고 <code>DefaultGame.ini</code> 에 <code>GameplayCueNotifyPaths</code> 를 추가했다. 그 뒤 조회 툴이 큐를 찾는지와 큐 재생은 따로 확인하지 않았다(플레이 중 문제 보고는 없음)
캡처에서 캐릭터가 구겨진 포즈	<code>CaptureViewport</code> 는 PIE 중에도 애니메이션이 없는 에디터 월드를 찍음	PIE 화면은 SlateInspector 위젯 스크린샷으로
SlateInspector로 보낸 키가 게임에 안 들어감	게임 뷰포트가 Slate 키보드 포커스를 받지 못함. 백그라운드 에디터는 약 3fps로 스로틀	입력이 필요한 검증은 사람 플레이로

우려했지만 문제가 없었던 것. 계획서는 "MCP로 만든 Enhanced Input 이벤트 노드가 바인딩 없이 조용히 안 불린다"는 보고를 경고했다. 이번 방식(노드 생성 → Started 핀 연결 → 전체 컴파일)에서는 5개 입력이 모두 정상 동작했다. GE 컴포넌트 같은 인스턴스드 서브오브젝트 배열과 IMC 매핑도 ObjectTools로 문제없이 편집됐다.

5 팀 프로젝트에 쓰는 방법 제안

이번 데모에서 MCP는 "정해진 사양을 에셋으로 옮기는 일"에 가장 강했다. 태그, 스탯 테이블, GameplayEffect, 큐, 입력 매핑, UI 배치, 레벨 배치처럼 속성 위주인 에셋은 전용 툴로 막힘 없이 만들어졌고, 넣은 값을 다시 읽어 검증할 수 있었다(단계 4의 GE 12개와 큐 3개에 약 6분. 사람 작업과 시간을 비교하지는 않았다). 규칙이 분명한 어빌리티 그래프도 DSL로 한 번에 썼다. 반대로 클래스 참조 변수, BP 인터페이스, 몽타주 노티파이처럼 **구조를 정하는 작업**은 전용 툴이 없어서 설계를 우회하거나 에디터 UI 자동화로 처리해야 했다. 이번에는 우회로 모두 해결했지만, 팀 작업에서는 사람이 먼저 만들어 두는 편이 안전하다. 또 MCP로는 PIE에 키 입력을 넣을 수 없어서 **입력이 필요한 동작 확인**은 사람이 플레이로 했다. 그래서 팀에서는 사람이 뼈대(클래스 변수, 인터페이스, 노티파이가 들어간 몽타주)와 사양 문서를 먼저 만들고, AI가 MCP로 데이터 에셋과 그래프를 채우는 분업을 권한다. 사람은 AI가 남긴 단계별 기록표와 커밋 단위로 검토하고 플레이로 확인한다.

운영 규칙

- **에디터 하나에 에이전트 하나.** 같은 에디터에 동시에 호출하면 응답이 섞인다. 병렬 작업은 에디터(또는 PC)를 나눠서 한다.
- **사양 문서를 먼저.** 수치, 태그 이름, 에셋 이름을 표로 정해 두면 AI가 그대로 만들고 철자까지 검증한다. 이번에도 태그 24개를 추가한 뒤 목록을 다시 읽어 확인했다.
- **단계마다 strict 컴파일, 다시 읽기, 커밋.** `read_graph_dsl` 은 티 이벤트 핀 같은 일부 연결을 보여 주지 않으니, 컴파일이 통과해도 핵심 연결은 `get_node_infos` 로 다시 읽는다. 되돌리기 쉽게 커밋을 잘게 나누고, 바이너리 에셋은 Git LFS로 관리한다.
- **공용·템플릿 에셋은 읽기만.** 읽기 툴만 써도 dirty가 될 수 있으니 저장 범위를 작업 폴더로 한정한다.
- **클래스 참조는 리터럴이나 사람이 만든 변수로.** CDO 참조로 우회하지 않는다.
- **기록을 남긴다.** "MCP로 됨 / 우회 / 사람 / 못 함"을 계속 쌓으면 어디까지 AI에 맡길지 팀이 판단할 근거가 된다.

맡기기 좋은 일과 사람이 할 일

AI + MCP에 맡기기 좋음	사람이 먼저 하거나 직접 할 일
태그, DataTable, CurveTable 같은 데이터 에셋의 대량 생성	게임플레이 구조 결정(베이스 클래스, 인터페이스, 클래스 변수)
GE 수치, 코스트·쿨타임, 큐 연결 같은 반복 설정	몽타주 노티파이, 애니메이션 편집
사양이 명확한 BP 함수와 어빌리티 그래프 초안	새 C++ 클래스(팀 합의가 필요한 영역)
IMC 매핑, UMG 트리과 레이아웃, 레벨 액터 배치	입력이 필요한 플레이 검증, 조작감과 밸런스 조정
컴파일·인스펙터·로그를 이용한 상태 점검	최종 저장 범위 확인과 푸시

다음에 해 볼 것

- Epic 공식 스킬 플러그인(`unreal-engine-skills-for-claude-code`)을 설치하고 같은 작업의 품질과 속도를 비교한다. 이번에는 설치되지 않아 에디터 내장 스킬(BlueprintBasicsSkill)만 썼다.
- 사람이 클래스 변수와 인터페이스 뼈대만 만든 상태에서 나머지를 MCP로 채우는 분업을 실제 기능 하나로 시험한다.
- MCP 서버 쪽에 PIE 입력 주입이나 몽타주 편집 툴이 추가되는지 핫픽스마다 `list_toolsets` 로 확인한다.

재현하려면

- UE 5.8(Templates and Feature Packs 포함)과 Visual Studio C++ 빌드 도구가 필요하다. C++ `DemoAttributeSet` 하나를 컴파일해야 하기 때문이다.

- Claude Code는 에디터와 같은 PC에서 돌린다(데스크톱 앱 Code 탭 로컬 세션 또는 CLI). MCP 서버가 127.0.0.1 에서만 열리므로 클라우드 세션은 붙을 수 없다.
- Third Person BP 템플릿으로 ActionDemo 를 만들 때 Variants에서 Combat, Platforming을 고른다. 그다음 DemoKit/install.ps1 -Build 를 실행하고 OpenEditor_EN.bat 으로 영어 에디터를 연다.
- 저장소 루트에서 Claude Code 세션을 열고 unreal-mcp 연결을 승인한 뒤, docs/demo/03-mcp-build-plan.md 순서대로 진행하라고 지시한다. 자세한 설치 절차는 docs/demo/01-setup-windows.md 에 있다.

참고 자료

docs/demo/01-setup-windows.md	PC 설치와 연결 방법
docs/demo/04-mcp-test-log.md	단계별 판정, 사용한 툴, 문제와 해결 기록
docs/demo/assets.md	템플릿 조사 결과와 계획 변경 요약
docs/done-action-demo-mcp.md	완료 문서(변경 파일, 검증, 주의사항)
docs/demo/screenshots/	PIE 화면, 에디터 캡처 비교
Git 브랜치	claude/elegant-einstein-ezzm7z 를 main 에 머지해 푸시했다(머지 커밋 8090fea)